📚 Clock Synchronization Protocol: Theory, Experiments, and Viva Prep

Clock synchronization across a network must contend with clocks that drift independently and with packets that do not spend equal time traveling each way. This lesson walks through the two-way timing exchange you implemented, the theory that asymmetry injects a bias of $(d_F - d_R)/2$, and the experiments you ran to show where the protocol holds up and where it starts to wobble.

Why clock synchronization is hard ⏱️

- **Clock drift**: each node's oscillator runs at a slightly different rate, so even if initially aligned they diverge over seconds.
- **Timestamp alone isn't enough**: a single timestamp reflects the remote clock *plus* the unknown forward-path delay.
- **Two failure modes**
  - **Delay asymmetry** – forward delay d_F ≠ reverse delay d_R, breaking the classic "symmetric-delay" assumption.
  - **Packet loss** – lost probes reduce the sample size and can bias the estimator if not handled.

> **Path asymmetry** – the difference between forward and reverse network propagation times, often expressed as $(d_F - d_R)/2$.

The two-way timing exchange 🔄

- **Step 1**: Client sends a **Sync** packet, stamping its local send time t_1 .
- **Step 2**: Server records arrival t_2, immediately replies with t_2 and its own send time t_3 .
- **Step 3**: Client receives reply at t_4 and now holds four timestamps: t_1, t_2, t_3, t_4 .

Computation

- Round-trip delay: $\Delta = (t_4 - t_1) - (t_3 - t_2)$.
- Clock offset estimate: $\theta = \frac{(t_2 - t_1) + (t_3 - t_4)}{2}$.

```
# minimal two-way exchange sketch
t1 = client.time()
send_sync(t1)
t2, t3 = server.reply()
t4 = client.time()
theta = ((t2 - t1) + (t3 - t4)) / 2
```

The formula assumes $d_F = d_R$; any mismatch appears as bias.

Where the $(d_F - d_R)/2$ offset comes from 📐

1. Write the true forward and reverse delays:
   - $t_2 = t_1 + d_F + \theta$
   - $t_4 = t_3 + d_R - \theta$
2. Insert into the symmetric estimator θ_{sym} and simplify:
   $\theta_{sym} = \theta + \frac{d_F - d_R}{2}$.
3. Thus the **bias** equals half the asymmetry; when $d_F = d_R$ the bias vanishes.

> **Asymmetric delay bias** – the systematic error introduced when forward and reverse path delays differ, equal to $(d_F - d_R)/2$.

The derivation shows why the classic NTP formula works only under the symmetric-delay assumption.

System architecture and tooling 🛠️

- **Ryu SDN controller** (Python 3.9 venv) installed via make_ryu_env.sh with:
  - ryu==4.34
  - eventlet==0.30.2
  - dnspython==1.16.0 (for optional DNS lookups)
- **Sync endpoints**: sync_server.py and sync_client.py use only the Python standard library (no external deps).
- **Experiment pipeline**:
  - numpy for generating delay/loss patterns and aggregating results.
  - matplotlib for the two result charts (offset vs. asymmetry; loss vs. probe loss & p95).

- **Network emulation**: tc netem injects controlled forward/reverse delays and per-direction loss rates.

The whole stack stays lightweight: the controller handles flow rules; the sync code stays pure Python; analysis runs in the host Python environment.

Experiment 1: offset error vs. path asymmetry 📈

- **Scenarios**:
  - sym_00 → no asymmetry (baseline).
  - asym_50 → forward 50 ms longer than reverse.
  - asym_100 → forward 100 ms longer than reverse.
- **Implementation**: tc netem delay 0ms 0ms on one direction and +Δ ms on the other, where Δ = 0, 25, 50 ms (half-asymmetry values).
- **Result**: Measured mean offset error lands exactly on the theoretical line $y = x$ (slope 1) – see *fig_delay_asymmetry.png*. Each data point (0,0), (25,25), (50,50) aligns with the dashed line, confirming the $(d_F - d_R)/2$ prediction.

> **Offset estimation error** – the absolute difference between the true clock offset and the value produced by the protocol, expressed in milliseconds.

Experiment 2: robustness to packet loss 📉

- **Loss model**: per-direction loss p set via tc netem loss X%.
- **Expected probe loss**: $1 - (1 - p)^2$ (probability that *either* forward or reverse probe is lost).
- **Observed**: Measured probe loss (blue line) tracks the black dashed theoretical curve almost perfectly (see *fig_loss.png*).
- **Clock error (p95)**: 95th-percentile error grows from ~200 µs at 0 % loss to ~450 µs at 40 % loss, a graceful degradation rather than a hard failure.

> **Probe loss** – the fraction of synchronization exchanges that fail because at least one direction's packet is dropped.
> **p95 clock error** – the value below which 95 % of offset estimates lie, a robust metric for worst-case performance.

Interpreting the result charts 🧭

- **Offset-vs-asymmetry chart**
  - *X-axis*: path asymmetry $(d_F - d_R)/2$ (ms).
  - *Y-axis*: mean offset error (ms).
  - Dashed line = theory; red points = measured (sym_00, asym_50, asym_100). Perfect overlap validates the bias formula.
- **Loss-vs-performance chart**
  - *Left Y-axis*: measured probe loss % (blue). Matches the black curve $1 - (1 - p)^2$, confirming the independence assumption.
  - *Right Y-axis*: p95 clock error µs (red). Shows a monotonic rise; the slope indicates how quickly error worsens with loss.
    Reading these graphs lets you point to concrete numbers ("at 30 % loss the probe loss is ≈ 51 % and p95 error ≈ 380 µs") when answering viva questions.

Viva prep: questions and crisp answers 🎙️

- **Q1: Derive the $(d_F - d_R)/2$ bias.**
  A: Show the four timestamps, express them with forward/reverse delays, substitute into the symmetric estimator, and simplify to reveal the extra term $(d_F - d_R)/2$.
- **Q2: Why does packet loss not crash the protocol?**
  A: Each exchange is independent; loss simply discards a sample. The estimator uses the remaining probes, so performance degrades gracefully, as shown by the p95 curve.
- **Q3: How does the experiment prove the theory?**
  A: The offset-error chart places measured points exactly on the $y = x$ line, confirming that error equals half the asymmetry for three distinct Δ values.
- **Q4: What are the limitations of your design?**
  A: It assumes a *fixed* offset during measurement, ignores clock drift within an exchange, and still relies on the symmetric-delay assumption for the basic estimator—only the bias term is corrected post-hoc.
- **Q5: Could you improve loss resilience?**
  A: Yes—by sending multiple probes per interval and applying a robust estimator (e.g., median of successful offsets) to reduce sensitivity to occasional drops.

Key Terms 🔑

Term	Definition
Path asymmetry	Difference between forward and reverse network propagation times, expressed as $(d_F - d_R)/2$.
Offset estimation error	Absolute deviation between the true clock offset and the protocol's estimated offset.
Probe loss	Percentage of synchronization exchanges lost because at least one direction's packet is dropped.
p95 clock error	95th-percentile of offset error, indicating worst-case performance for most runs.
Symmetric-delay assumption	The simplification that forward and reverse path delays are equal, underlying the classic NTP formula.